

UD 01

PROGRAMACIÓN DE PROCESOS

PROGRAMACIÓN DE SERVICIOS Y PROCESOS

CFGS DESARROLLO DE APLICACIONES MULTIPLATAFORMA

v1.1



1.- INTRODUCCIÓN

Cuando nos manejamos en ciertos niveles de conocimiento, debemos conocer la diferencia entre acepciones que, aunque contextualmente en ocasiones sean sinónimas, no siempre lo son.

Términos como “aplicación”, “proceso”, “programa”, “ejecutable”... en ocasiones pueden ser sinónimos, pero en términos técnicos tienen diferencias sustanciales entre ellos.

En la presente unidad didáctica aprenderemos a diferenciar estos términos. Conoceremos asimismo cómo se gestionan los procesos dentro del SO. Veremos algoritmos de planificación. Hablaremos de programación concurrente, comunicación entre procesos y su sincronización.

2.- CONCEPTOS BÁSICOS

Lo primero, tal como hemos comentado en la introducción, será aprender a diferenciar algunos términos utilizados frecuentemente como sinónimos aunque realmente no lo sean.

Programa: Conjunto comprendido por código y datos (conjunto de instrucciones) que se encuentra en el disco que realizan de manera automática tareas específicas, no ofreciendo en ocasiones ninguna interfaz gráfica al usuario (GUI) y no están destinadas al usuario final.

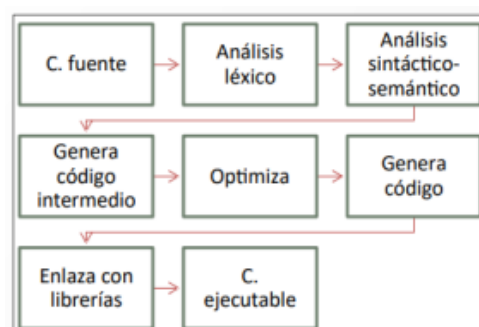
Aplicación: Tipo de programa informático desarrollado como herramienta para resolver automáticamente un problema específico.

Una aplicación es un programa, pero no todos los programas son aplicaciones, una aplicación necesita de otros programas para funcionar.

Una aplicación es un programa o colección de programas destinados al usuario final para realizar una tarea o lograr un propósito.

La línea entre el término aplicación y programa se ha desdibujado con los años y se usan los términos indistintamente, aunque realmente no lo son.

Ejecutable: Fichero que contiene la información necesaria para poder crear el proceso correspondiente. Este fichero será ejecutado en un ordenador.



Teniendo clara la diferencia entre aplicación y ejecutable... ¿qué es un proceso?

Proceso: De manera MUY resumida, un proceso es un programa en ejecución. Si queremos ser un poco más exactos, debemos decir que es el estado de un programa en un momento concreto de su ejecución, lo que incluye:

- **Contador de programa:** indica el punto de ejecución
- **Imagen de memoria:** todo el espacio en memoria que está utilizado
- **Estado del procesador:** valor de los registros del procesador que se están utilizando

Un proceso existe mientras que se está ejecutando una aplicación. Es más, la ejecución de una aplicación puede implicar que se arranquen varios procesos en nuestro equipo.

Cuando se interrumpe la ejecución de un proceso por alguna causa, se verán más adelante, se guarda una copia de los tres elementos nombrados arriba para poder restaurar la ejecución del proceso en el mismo punto en el que se interrumpió.

Los procesos son entidades independientes aunque ejecuten el mismo programa de manera que pueden coexistir en ejecución dos procesos que ejecutan el mismo programa. Por ejemplo se pueden tener abiertos dos archivos pdf a la vez o si se utiliza el navegador chrome cada pestaña que se tiene abierta es un proceso diferente.

¿Y los Sistemas Operativos qué son?

Sistema Operativo: Es un programa encargado de gestionar de manera eficiente todos los recursos hardware y software del equipo proporcionando un entorno amigable y fácil de usar al usuario.



Demonio: proceso que se ejecuta en segundo plano no accesible al usuario, su función es proporcionar un servicio al resto de procesos que se ejecutan. Un ejemplo de demonio sería el Garbage collector de Java que se encarga de liberar memoria cuando un objeto ya no tiene ninguna referencia a memoria activa.

3.- GESTIÓN DE PROCESOS

Como sabemos, un microprocesador es capaz de ejecutar miles de millones de instrucciones por segundo. Como ejemplo de la evolución que han sufrido, desde el Intel 8080 (lanzado en 1974) que realizaba 640 KIPS hasta el AMD Ryzen Threadripper 3990X (lanzado en 2020) con 2.356.230 MIPS. A pesar de no ser el último ni más potente procesador del mercado, podemos apreciar la diferencia de potencia de los procesadores.

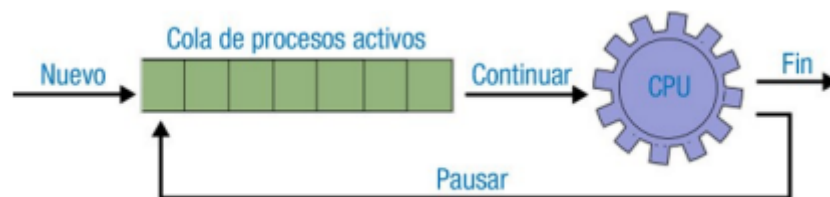
Como se ha dicho anteriormente un proceso es el estado de un programa en un momento dado de su ejecución con todo lo que ello significa: contador, memoria y registros de la CPU. Es el SO el encargado de poner en ejecución y gestionar los procesos.

Los procesos para poder cumplir con la multiprogramación y programación paralela pasan por diferentes estados a lo largo de su ciclo de vida. Los cambios de estado también son gestionados por el SO. Existen varios modelos para implementar el ciclo de vida de un proceso.

3.1.- ESTADO DE UN PROCESO

El uso del microprocesador debe repartirse entre los distintos procesos. Los nuevos microprocesadores con varios núcleos favorecen la concurrencia de procesos ya que, recordemos, aparte de las aplicaciones que nosotros estemos ejecutando, también se está ejecutando por debajo el SO con sus correspondientes procesos.

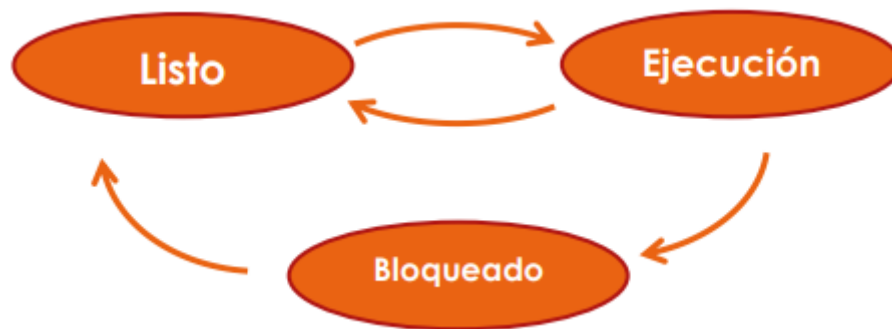
En el siguiente esquema podemos apreciar un esquema muy básico de cómo podemos planificar la ejecución de varios procesos en una CPU.



En este modelo podemos ver cómo se gestionan los procesos:

1. Los procesos nuevos se meten en la cola de procesos activos en el sistema, diremos que el proceso está “preparado” para ser ejecutado por la CPU.
2. Los procesos avanzan en esa cola en función del planificador de procesos utilizado por el SO.
3. Cada proceso se mantendrá en la CPU una cantidad de tiempo X, que vendrá definida de igual manera por el planificador de procesos utilizado. Si el proceso no ha finalizado su requerimiento de CPU, volverá a la cola de procesos preparados.
4. Cuando un proceso finaliza, sale del sistema de gestión de procesos.

En este caso, se ha utilizado un modelo muy simplificado de 3 estados:



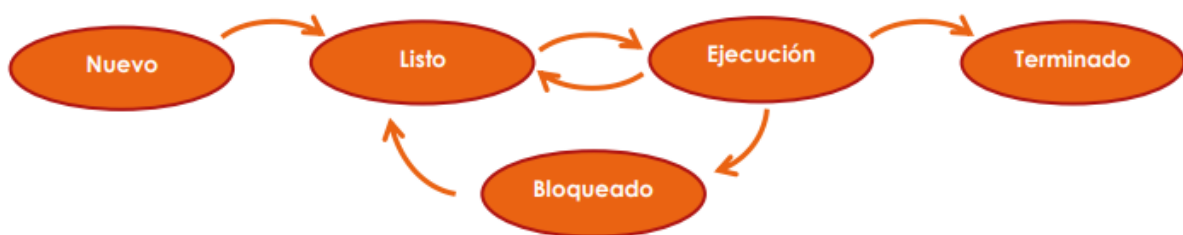
Listo: Puede entrar en la CPU, ya tiene asignada una zona en memoria

Ejecución: Se encuentra dentro de la CPU ejecutando sus instrucciones

Bloqueado: Sin posibilidad de entrar a la CPU, por ejemplo, está esperando una operación de E/S

No obstante, este modelo de 3 estados está muy simplificado y, aunque no es muy cómodo para entender de manera superficial la forma de funcionar, no es real.

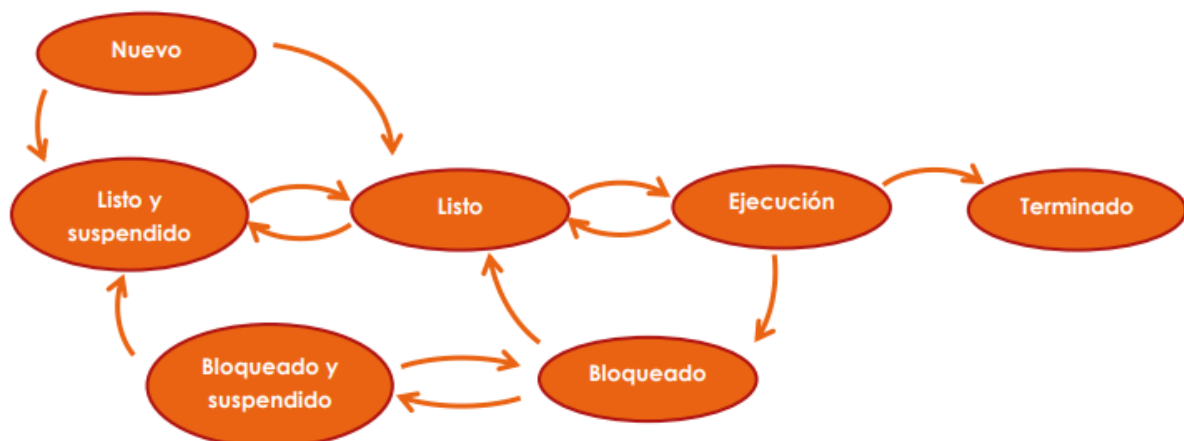
Al modelo de 3 estados, incorporaremos 2 nuevos estados.



Nuevo: Se está creando o acaba de crearse, pero no está listo para la ejecución, por ejemplo, si ya ha alcanzado el límite de memoria.

Terminado: Ha finalizado y libera su imagen de memoria, también pasa a terminado si se produce un error y deja de ejecutarse.

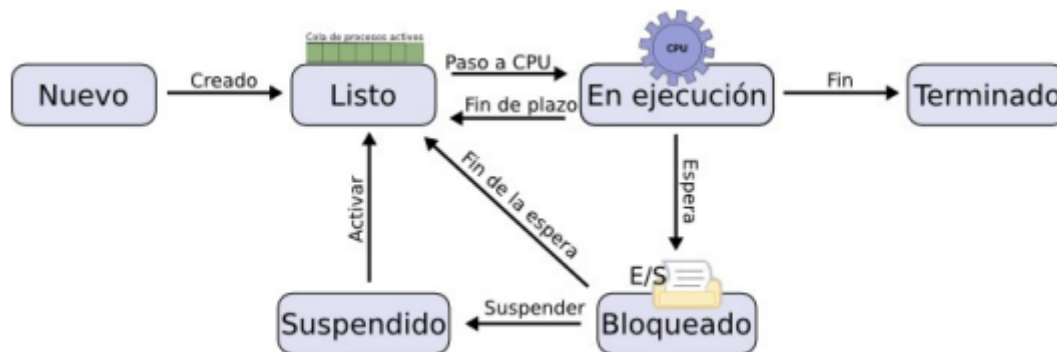
No obstante, el modelo más real que podemos encontrar es el modelo de 7 estados.



Bloqueado y suspendido: Procesos bloqueados que se encuentran esperando en memoria secundaria (RAM) al no haber más procesos en memoria.

Listo y suspendido: Procesos preparados para entrar en ejecución pero que se encuentran en memoria secundaria al no haber más procesos en memoria.

Los estados “Listo y suspendido” y “Bloqueado y suspendido” suelen agruparse en el mismo lugar:



3.2.- COLAS DE PROCESOS

Como uno de los objetivos del SO es la multiprogramación, el SO necesita saber qué procesos están en cada estado para saber cuál de ellos puede entrar en la CPU, para esto hace uso de colas de procesos.

- Cola general de procesos: contiene todos los procesos del sistema
- Cola de procesos preparados: contiene aquellos procesos listos para ejecutarse
- Colas de dispositivo: contiene los procesos que están esperando alguna operación de E/S

3.3.- PLANIFICACIÓN DE PROCESOS

Si hubiera un sólo proceso ejecutándose en el sistema, no sería necesario planificar nada, ni tan siquiera tener en cuenta las colas, pero sabemos que la realidad no es esa. Como hemos comentado, hay miles de procesos ejecutándose o esperando ser ejecutados por el procesador en un momento dado, entonces... ¿Quién decide qué proceso se ejecuta? ¿En base a qué toma esa decisión?

El SO necesita un planificador de procesos que se encargue de gestionar las colas de procesos. Existen dos tipos de planificación:

- Corto plazo: se encarga de elegir cuál es el siguiente proceso en entrar a la CPU, esta operación se ejecuta muchas veces y cada poco tiempo (milisegundos) por lo que el algoritmo ha de ser sencillo. Existen tres tipos de algoritmos para esta planificación.

- Sin desalojo: solo se cambia de proceso en ejecución si este se termina o bloquea.
- Con desalojo: si llega un proceso con más prioridad se desaloja al que está ejecutándose para que entre ese proceso prioritario.
- Tiempo compartido: cada cierto tiempo se cambia el proceso en ejecución.
- Largo plazo: se encarga de gestionar los procesos que pasan a la cola de preparados, se invoca con poca frecuencia.

Cada proceso es único e impredecible, los hay orientados a CPU y orientados a E/S.

CPU	E/S	CPU	E/S	CPU	CPU	E/S	CPU	E/S	CPU	E/S	CPU
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Los algoritmos de programación a corto plazo deben de cumplir los siguientes objetivos:

Justicia/eqüidad: todos los procesos han de tener su turno de CPU.

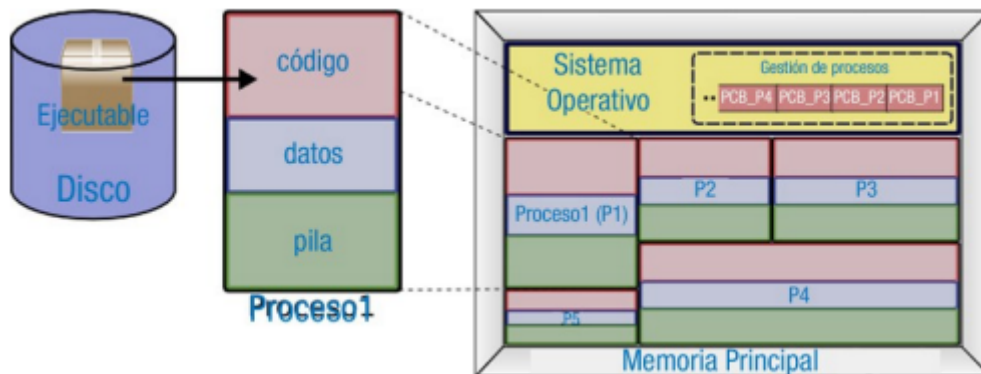
Eficiencia/utilización: deben mantener la CPU ocupada el mayor tiempo posible.

Hemos visto que un proceso, desde su creación hasta su fin (durante su vida), pasa por muchos estados. Esa transición de estados es transparente para él, todo lo realiza el SO. Desde el punto de vista de un proceso, él siempre se está ejecutando en la CPU sin esperas. Dentro de la gestión de procesos vamos a destacar dos componentes del SO que llevan a cabo toda la tarea: el cargador y el planificador.

El cargador es el encargado de crear los procesos. Cuando se inicia un proceso (para cada uno), el cargador realiza las siguientes tareas:

1. **Carga el proceso en memoria principal.** Reserva un espacio en la RAM para el proceso. En ese espacio copia las instrucciones del fichero ejecutable de la aplicación, las constantes y deja un espacio para los datos (variables) y la pila (llamadas a funciones). Un proceso, durante su ejecución, no podrá hacer referencia a direcciones que se encuentren fuera de su espacio de memoria; si lo intentara, el SO lo detectará y generará una excepción (produciendo, por ejemplo, los típicos pantallazos azules de Windows).
2. **Crea una estructura de información llamada PCB** (Bloque de Control de Proceso). La información del PCB es única para cada proceso y permite controlarlo. Esta información también la utilizará el planificador. Entre otros datos, el PCB estará formado por:
 - **Identificador del proceso o PID.** Es un número único para cada proceso, como un DNI de proceso.
 - **Estado actual del proceso:** en ejecución, listo, bloqueado, suspendido, finalizando.

- **Espacio de direcciones de memoria** donde comienza la zona de memoria reservada al proceso y su tamaño.
- **Información para la planificación:** prioridad, quantum, estadísticas, ...
- **Información para el cambio de contexto:** valor de los registros de la CPU, entre ellos el contador de programa y el puntero a pila. Esta información es necesaria para poder cambiar de la ejecución de un proceso a otro.
- **Recursos utilizados.** Ficheros abiertos, conexiones, ...



3.4.- CAMBIO DE CONTEXTO

Se llama contexto al conjunto de datos relacionados con el proceso en ejecución, dentro de esos datos se encuentra el estado del proceso, el estado del procesador (valores de los registros) y el espacio de memoria que ocupa el proceso.

Cuando se cambia el proceso en ejecución se produce un cambio de contexto mediante el cual se guarda una copia del contexto del proceso que se interrumpe para poder más tarde volver al mismo punto de ejecución del mismo.

Los cambios de contexto son tiempos perdidos ya que el procesador no realiza ninguna tarea útil durante ese tiempo. La duración depende de la arquitectura del procesador

Imaginemos que, en nuestro equipo, en un momento determinado, podemos estar escuchando música, editando un documento, al mismo tiempo, programando y navegando en Internet. En este caso, tendremos ejecutándose en el sistema cuatro aplicaciones distintas, que pueden ser: el reproductor multimedia VLC, el editor de textos writer de OpenOffice, el Visual Studio Code y el navegador Firefox. Todos ellos ejecutados sin fallos y cada uno haciendo uso de sus datos.

El sistema operativo (el planificador), al realizar el cambio una aplicación a otra tiene que guardar el estado en el que se encuentra el microprocesador y cargar el estado en el que estaba el microprocesador cuando cortó la ejecución de otro proceso para continuar con ese. Pero, ¿qué es el estado de la CPU?

Una CPU, además de circuitos encargados de realizar las operaciones con los datos (llamados circuitos operacionales), tiene unos pequeños espacios de memoria (llamados registros), en los que se almacenan temporalmente la información que, en cada instante, necesita la instrucción que esté procesando la CPU. El conjunto de registros de la CPU es su estado.

Entre los registros destacamos el Registro Contador de Programa y el puntero a la pila.

- **El Contador de Programa**, en cada instante almacena la dirección de la siguiente instrucción a ejecutar. Recordemos, que cada instrucción a ejecutar, junto con los datos que necesite, es llevada desde la memoria principal a un registro de la CPU para que sea procesada; y, el resultado de la ejecución, dependiendo del caso, se vuelve a llevar a memoria (a la dirección que ocupe la correspondiente variable). Pues el Contador de Programa, apunta a la dirección de la siguiente instrucción que habrá que traer de la memoria, cuando se termine de procesar la instrucción en curso. Este Contador de Programa nos permitirá continuar en cada proceso por la instrucción en dónde lo hubiéramos dejado todo.
- **El Puntero a Pila**, en cada instante apunta a la parte superior de la pila del proceso en ejecución. En la pila de cada proceso es donde será almacenado el contexto de la CPU. Y de donde se recuperará cuando ese proceso vuelva a ejecutarse.

